

React Hooks — Interview Notes

Short definitions • interview answers • code patterns • real-world examples

Core idea: Hooks are special React functions that let functional components use features such as state, effects, context, refs, and reusable stateful logic.

Rules of Hooks: Call Hooks only at the top level of a React function component or custom Hook. Do not call them inside loops, conditions, or nested functions. Hook names conventionally begin with use.

1. useState()

Definition: useState lets a functional component store a value that can change over time. It returns the current state and a setter function.

Syntax:

```
const [count, setCount] = useState(0);
```

Interview answer: “useState is a React Hook used to manage local component state. When its setter is called, React schedules a re-render so the UI can reflect the new state.”

Real-world example: A shopping cart stores the number of items. Clicking “Add to cart” calls setCartCount(cartCount + 1), and React updates the displayed count.

```
const [cartCount, setCartCount] = useState(0);

function addToCart() {
  setCartCount(cartCount + 1);
}
```

Important: Do not directly mutate state such as count = count + 1. Use the setter.

2. useEffect()

Definition: useEffect synchronizes a component with an external system or performs side effects after rendering, such as fetching data, subscriptions, timers, or updating the document title.

Interview answer: “useEffect lets a component perform side effects after React renders. The dependency array controls when the effect runs, and a cleanup function can undo subscriptions, timers, or other resources.”

Real-world example: On a product page, fetch product details when the product ID changes. If a WebSocket subscription is created, clean it up when the component unmounts or the dependency changes.

```
useEffect(() => {
  // side effect
  return () => { /* cleanup */ };
}, [dependencies]);
```

3. useContext()

Definition: useContext reads a value from React Context so shared data can be accessed without manually passing it through every intermediate component.

Interview answer: “useContext is useful for shared values such as theme, authenticated user information, or locale when many components need access to the same context.”

Real-world example: A website theme switcher provides light/dark mode through context. Navbar, sidebar, and buttons can read the current theme without prop drilling.

```
const theme = useContext(ThemeContext);
```

4. useRef()

Definition: useRef returns a mutable ref object whose current value persists across renders. Changing it does not itself trigger a re-render. It can also reference a DOM element.

Interview answer: “I use useRef when I need a persistent mutable value or a direct reference to a DOM node, without needing a re-render when the value changes.”

Real-world example: A login form automatically focuses the email input when the page opens.

```
const inputRef = useRef(null);

<input ref={inputRef} />

inputRef.current?.focus();
```

5. useMemo()

Definition: useMemo memoizes the result of a calculation and recomputes it when dependencies change. It is a performance optimization.

Interview answer: “useMemo can avoid repeating an expensive calculation when its inputs have not changed. I use it when the calculation is costly or profiling shows a benefit.”

Real-world example: Filtering and sorting thousands of products based on search and category; memoizing the derived list can avoid repeating the calculation during unrelated re-renders.

```
const filteredProducts = useMemo(() => {
  return products.filter(p => p.name.includes(search));
}, [products, search]);
```

6. useCallback()

Definition: useCallback memoizes a function reference until its dependencies change.

Interview answer: “useCallback is useful when passing callbacks to memoized child components or when a stable function reference matters. It can prevent unnecessary child re-renders caused by a new function identity.”

Real-world example: A parent passes a selection handler to a memoized product-row component; a stable callback can help avoid unnecessary rendering.

```
const handleSelect = useCallback((id) => {
  setSelectedId(id);
}, []);
```

7. Custom Hooks

Definition: A custom Hook is a JavaScript function, conventionally starting with use, that reuses stateful React logic across components.

Interview answer: “I create a custom Hook when multiple components need the same stateful behavior or side-effect logic. It improves reuse and keeps components focused on UI.”

Real-world example: Several pages need online/offline status. Create useOnlineStatus() and reuse the logic.

```
function useOnlineStatus() {
  const [online, setOnline] = useState(navigator.onLine);
  // add effect/listeners here
  return online;
}
```

Quick Interview Comparison

Hook	Purpose	Real-world example
useState	Manage local state	Cart count
useEffect	Side effects / synchronization	Fetch data / subscription
useContext	Read shared context	Theme / user
useRef	Persistent value / DOM reference	Focus input
useMemo	Memoize computed value	Expensive filtering
useCallback	Memoize function reference	Stable child callback
Custom Hook	Reuse stateful logic	useOnlineStatus

Common Interview Questions

What are Hooks?

Hooks are special React functions that let function components use React features such as state and effects.

Why were Hooks introduced?

They allow stateful logic and side effects in function components and make logic easier to reuse without class components.

State vs normal variable?

Changing a normal variable does not tell React to update the UI. Updating state through its setter schedules a re-render.

What is the dependency array in useEffect?

It specifies the reactive values the effect depends on; the effect re-runs when those dependencies change.

Does useRef cause a re-render?

No. Updating ref.current does not by itself trigger a re-render.

useMemo vs useCallback?

useMemo memoizes a calculated value; useCallback memoizes a function reference.

Can Hooks be called inside if/else or loops?

No. Hooks should be called at the top level so React can preserve Hook call order across renders.

When should you create a custom Hook?

When the same stateful or side-effect logic is needed by multiple components.

Interview-ready 30-second answer

“React Hooks are special functions that let function components use React features. Common Hooks include useState for state, useEffect for side effects and synchronization, useContext for shared context values, and useRef for persistent mutable values or DOM references. useMemo and useCallback are performance-oriented memoization Hooks, while custom Hooks let us reuse stateful logic. Hooks must be called at the top level of React components or other Hooks.”

Study tip: Understand the problem each Hook solves first; then practice its syntax with one small real-world example. You do not need to memorize every Hook API before building projects.